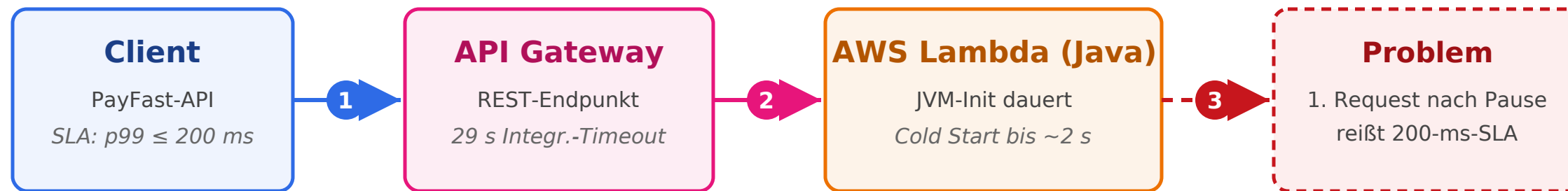


Battle Card 4 — Lambda Cold Start: SnapStart vs. Provisioned Concurrency

Java-API leidet unter Cold Starts — Latenz-SLA von 200 ms einhalten



Zwei Lösungen — je nach Traffic-Profil:

A · Lambda SnapStart

Snapshot der initialisierten Umgebung, Restore statt Neustart (Java 11+, Python 3.12+, .NET 8). Cold Start ~2 s → einige Dutzend ms.

Kosten: keine Standing-Kosten → ideal bei spikey / schwankendem Traffic. *Kleiner Restore-Overhead bleibt; Snapshot-sichere Init nötig.*

Merke: unregelmäßiger Traffic + Kosten sparen → SnapStart

B · Provisioned Concurrency

Hält N Umgebungen dauerhaft vorgewärmt → 0 ms Cold Start, sofort antwortbereit für die konfigurierte Concurrency.

Kosten: zahlt 24/7 pro vorgehaltener Umgebung (Standing Cost). *Per Application Auto Scaling nach Zeitplan/Metrik anpassbar.*

Merke: planbare Dauerlast + härtestes SLA (0 ms) → Provisioned

Merksätze: Cold Start = JVM-Init-Strafe · SnapStart = Snapshot/Restore, keine Standing-Kosten · Provisioned Concurrency = dauerhaft warm, 0 ms, 24/7-Kosten